

MODULE 1

27 November 2025 23:49

Module:1 Database Systems Concepts and Architecture

Need for database systems – Characteristics of Database Approach – Advantages of using DBMS approach - Actors on the Database Management Scene: Database Administrator - Classification of database management systems - Data Models - Schemas and Instances - Three-Schema Architecture - The Database System Environment - Centralized and Client/Server Architectures for DBMSs – Overall Architecture of Database Management Systems

The Need for Database Systems - Why We Moved Beyond Files

- * **Imagine running VIT-Chennai with separate, disconnected files for the schools, library, registrar, accounts, and hostel.**

One of you want to change your phone number/address.

- ☹ How many files need updating?
- ☹ What if updates are missed?

Welcome to the world before databases.

- ▶ In traditional approach, information is stored in flat files which are maintained by the file system under the operating system's control.
- ▶ Application programs go through the file system in order to access these flat files.

Problems with traditional approach for storing data:

1. Data Redundancy & Inconsistency:

- ⇒ The same data (e.g., student address) stored in multiple files (Registrar, Hostel, Accounts, School).
- ⇒ Wastes storage and, more critically, leads to multiple updates.
- ⇒ If not updated everywhere, data becomes inconsistent. Which address is correct?

2. Data Isolation:

- ✧ Data is scattered in non-standard formats.
- ✧ Writing a new application (e.g., "Alumni Portal") to get a combined view is extremely difficult.

3. Data Integrity:

- ◆ Enforcing rules (e.g., "Student GPA must be between 0 and 4.0") is hard.
- ◆ Rules are buried in application code, not centralized. Change is a nightmare.

4. Concurrent Access Anomalies:

- ▶ Two librarians try to issue the last copy of a book to two different students simultaneously. Without control, both might succeed → Logical Error.

5. Atomicity Problems:

- ◇ A "fee payment" involves deducting from account and updating payment status. If the system crashes in between, the system is left in an inconsistent state (money gone but status 'unpaid').

6. Security Issues:

- Difficult to provide granular access. You either give access to the entire student.txt file or none at all. Cannot easily hide salary details from the library clerk.

7. Lack of Data Abstraction:

- Applications are dependent on the physical structure of files (record length, ordering). Changing the file structure breaks all applications.

These problems aren't just **inconveniences**;
they are **business risks**.

We needed a paradigm shift.

⇒ Enter the **Database Approach**

What is Database ?

It is a collection of inter-related data which is used to retrieve, insert and delete the data efficiently. It is also used to organize the data in the form of a table, schema, views, and reports, etc.

For example: VIT-Chennai's Database organizes the data about the admin, staff, students and faculty etc. Using the database, you can easily retrieve, insert, and delete the information

Database Management System

DBMS (Database Management System) is a software system that manages, stores, and retrieves data efficiently in a structured format. For example: MySQL, Oracle.

- It allows users to create, update, and query databases efficiently.
- Ensures data integrity, consistency, and security across multiple users and applications.
- Reduces data redundancy and inconsistency through centralized control.
- Supports concurrent data access, transaction management, and automatic backups

DBMS allows users the following tasks:

Data Definition: It is used for [creation](#), [modification](#), and [removal](#) of definition that defines the organization of data in the database.

Data Updation: It is used for the [insertion](#), [modification](#), and [deletion](#) of the actual data in the database.

Data Retrieval: It is used to [retrieve](#) the data from the database which can be used by applications for various purposes.

User Administration: It is used for [registering](#) and [monitoring users](#), [maintain](#) data integrity, [enforcing](#) data security, [dealing](#) with concurrency control, [monitoring](#) performance and [recovering](#) information corrupted by unexpected failure.

Characteristics of DBMS

→ It uses a digital repository established on a server to store and manage the

information.

- It can provide a clear and logical view of the process that manipulates data. DBMS contains automatic backup and recovery procedures.
- It contains ACID properties which maintain data in a healthy state in case of failure.
- It can reduce the complex relationship between data.
- It is used to support manipulation and processing of data. It is used to provide security of data.
- It can view the database from different viewpoints according to the requirements of the user.

Difference Between File System and DBMS

Basics	File System	DBMS
Structure	The file system is a way of arranging the files in a storage medium within a computer.	DBMS is software for managing the database.
Data Redundancy	Redundant data can be present in a file system.	In DBMS there is no redundant data.
Backup and Recovery	It doesn't provide inbuilt mechanism for backup and recovery of data if it is lost.	It provides in house tools for backup and recovery of data even if it is lost.
Query processing	There is no efficient query processing in the file system.	Efficient query processing is there in DBMS.
Consistency	There is less data consistency in the file system.	There is more data consistency because of the process of normalization .
Complexity	It is less complex as compared to DBMS.	It has more complexity in handling as compared to the file system.
Security Constraints	File systems provide less security in comparison to DBMS.	DBMS has more security mechanisms as compared to file systems.
Cost	It is less expensive than DBMS.	It has a comparatively higher cost than a file system.
Data Independence	There is no data independence.	In DBMS data independence exists, mainly of two types: 1) Logical Data Independence . 2)Physical Data Independence.
User Access	Only one user can access data at a time.	Multiple users can access data at a time.

Meaning	The users are not required to write procedures.	The user has to write procedures for managing databases
Sharing	Data is distributed in many files. So, it is not easy to share data.	Due to centralized nature data sharing is easy
Data Abstraction	It give details of storage and representation of data	It hides the internal details of Database
Integrity Constraints	Integrity Constraints are difficult to implement	Integrity constraints are easy to implement
Attributes	To access data in a file , user requires attributes such as file name, file location.	No such attributes are required.
Example	Cobol , C++	Oracle , SQL Server

A file system manages storage, while a DBMS provides efficient data management.

Advantages of DBMS

- 😊 **Controls database redundancy:** It can control data redundancy because it stores all the data in one single database file and that recorded data is placed in the database.
- 😊 **Data sharing:** In DBMS, the authorized users of an organization can share the data among multiple users.
- 😊 **Easy Maintenance:** It can be easily maintainable due to the centralized nature of the database system.
- 😊 **Reduce time:** It reduces development time and maintenance need.
- 😊 **Backup:** It provides backup and recovery subsystems which create automatic backup of data from hardware and software failures and restores the data if required.
- 😊 **Multiple user interface:** It provides different types of user interfaces like graphical user interfaces, application program interfaces

Disadvantages of DBMS

- 😞 **Cost of Hardware and Software:** It requires a high speed of data processor and large memory size to run DBMS software.
- 😞 **Size:** It occupies a large space of disks and large memory to run them efficiently.

- ☹ **Complexity:** Database system creates additional complexity and requirements.
- ☹ **Higher impact of failure:** Failure is highly impacted the database because in most of the organization, all the data stored in a single database and if the database is damaged due to electric failure or database corruption then the data may be lost forever.

The Database System Environment

- The database system environment encompasses the components and interactions involved in managing and utilizing data.
- A DBMS is a complex software system designed to handle the creation, retrieval, and management of data.
- It serves as the interface between the database and its users, ensuring data consistency and availability.
- E-commerce companies rely heavily on complex DBMS environments.
- The interaction between catalog databases, customer management systems, and transaction processing ensures a smooth user experience.
- Banks use DBMS environments to maintain customer accounts, transactions, and loan data, etc.

DBMS Environment - Key Components and Functions

At its core, the DBMS environment includes the following key components:

- **Database:** The actual data storage, this is nothing but the disk drives.
- **DBMS Catalog** – Contains metadata, such as schema structures, data types, and mappings etc.

- **Operating System (OS)** – Manages low-level operations like reading and writing data from disks.
- **Buffer Management** – This is needed to enhance the performance by scheduling disk I/O operations.

Let's understand this through an **example**:

In an online bookstore, the database stores customer information, book inventories, and sales records. The buffer management gives rapid response when a customer searches for a book by minimizing disk reads.

DBMS Environment - Users and Their Interfaces

Different users interact with a DBMS. Each of them use the DBMS in a different way, which is why they need unique interfaces to perform their roles effectively.

Database Administrators (DBAs)

DBAs are responsible for managing the database and ensuring it runs smoothly. They use privileged commands to create and modify database structures, set user permissions, monitor database health and performance.

Example – In a financial institution, a DBA can adjust database schemas to accommodate new banking products or grant access to new employees.

Casual and Interactive Users

Casual users access the database sporadically and typically through query interfaces. And they might use simple query languages or forms to fetch information.

Example – A marketing manager at a company may use an interactive dashboard to pull customer demographics for a campaign analysis.

Application Programmers

Professionals write programs in host languages like Java or Python that interact with the database. They rely on pre-compilers that extract Data Manipulation Language (DML) commands and convert them into executable code.

Example – A programmer developing a retail app may write a script in Java that fetches product details from the database and displays them on the app's interface.

Parametric Users

Parametric users such as data entry clerks use applications with predefined transactions. These users input data or select parameters that run canned transactions repeatedly.

Example – Bank tellers input transaction details like account numbers and withdrawal amounts into an application that communicates with the DBMS to process transactions.

Core DBMS Component Modules

Let us now take a look at the major DBMS key modules, each fulfilling specific functions:

DDL Compiler

- ▶ Data Definition Language (DDL) compiler [processes schema definitions and updates](#) the DBMS catalog with metadata.
- ▶ It updates schema changes, like creating tables or modifying columns, are reflected in the catalog etc.

Example – When a company expands its product line, the DBA uses the DDL compiler to modify existing tables or add new ones to store additional product

details.

Query Compiler and Optimizer

- ▶ The query compiler **parses and validates user queries, translating them into an internal format**. This optimized query structure is then processed by the query optimizer.
- ▶ The optimizer ensures that the **most efficient execution plan** is used.

Example – If a user runs a query to find all orders shipped last month, the optimizer gives the user the best index to retrieve data quickly.

Runtime Database Processor

The runtime database processor gives the transactions and interacts with the system catalog. It is used for real-time data processing. This module supports data consistency and manages the main memory buffer.

Example – When an e-commerce site updates product stock levels after a sale, the runtime database processor updates the database and the in-memory buffers.

Concurrency Control and Backup Systems

Backup systems are critical for ensuring data integrity and recovery.

- **Concurrency Control** – Manages simultaneous database access to prevent data conflicts.
- **Backup System** – Regularly creates copies of the database to safeguard against data loss.

Example – During peak shopping hours, there are multiple customers may place orders simultaneously. Concurrency control helps to maintain stock levels are updated accurately without overlaps or errors.

Essential DBMS Utilities

DBMS utility programs help DBAs manage and maintain databases efficiently.

Here are some vital utilities:

- **Data Loading Utility:** The loading utility transfers the data from external files into the database. It is often converting formats in the process.
- **Backup and Restore Utilities:** The backup utility is used for a complete copy of the database is available in case of failure, while the restore utility reverts to the backup to recover lost data.
- **Performance Monitoring Tools:** These tools analyze the database usage patterns and provide statistics. The DBAs use this data to make decisions about indexing. The file reorganization, and system upgrades.

DBMS Environment – IDEs and Tools

Another important part of the system is the development tools that streamline the creation of database applications. Examples include:

- **CASE Tools: CASE (Computer-Aided Software Engineering)** tools are used for database design by automating parts of the process. This is helping designers maintain consistency. For example, a tech firm uses CASE tools to ensure their database schema matches the evolving needs of their SaaS product.
- **Integrated Development Environments (IDEs):** Development environments like **PowerBuilder** or **JBuilder** simplify GUI design. Also simplifies coding, and testing of database applications. A developer using PowerBuilder may create a front-end interface for a medical records system that interacts seamlessly with a backend DBMS.

DBMS Environment – Communication and Connectivity

DBMSs are designed to connect with remote users and integrate with other systems through network interfaces and middleware. This is especially critical for distributed databases:

- **Network Software** – Allows remote access to the DBMS.
- **Middleware** – Acts as a bridge, facilitating communication between the client applications and the DBMS.

For example, a distributed retail chain may have a central database that stores product data. Each outlet accesses it through network connections.

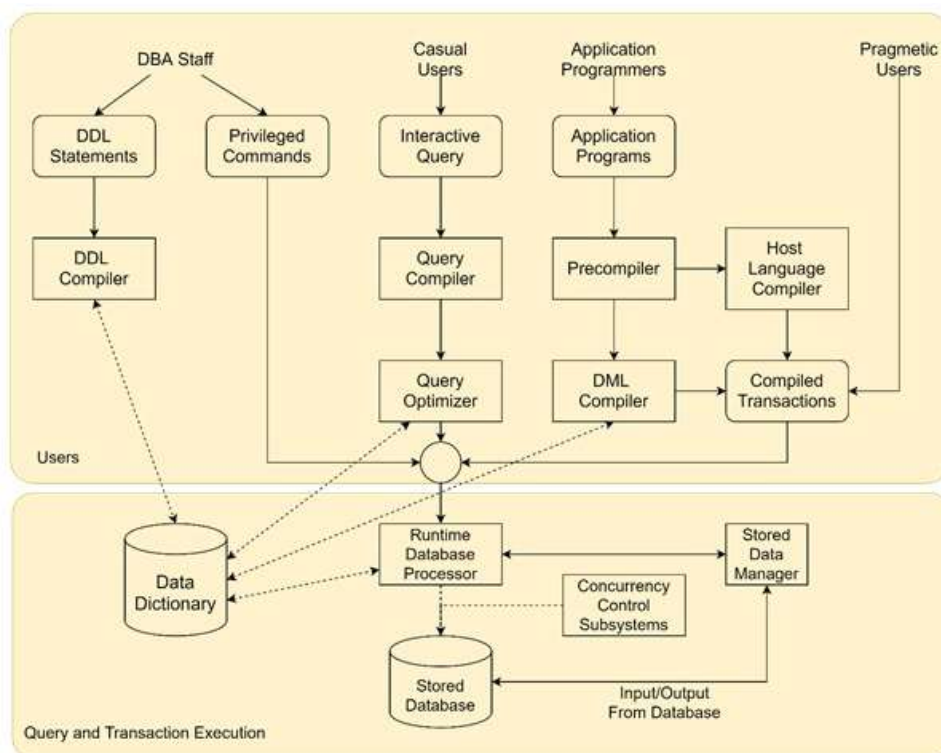


Figure 1: Component modules of a DBMS and their interactions.

The above figure illustrates, in a simplified form, the typical **DBMS components**.

- ▶ The figure is divided into two parts. The top part of the figure refers to the various users of the database environment and their interfaces.
- ▶ The lower part shows the internal modules of the DBMS responsible for storage of data and processing of transactions.

- ▶ The database and the DBMS catalog are usually stored on disk. Access to the disk is controlled primarily by the **operating system (OS)**, which schedules disk read/write. Many DBMSs have their own **buffer management** module to schedule disk read/write, because management of buffer storage has a considerable effect on performance.
- ▶ A higher-level **stored data manager** module of the DBMS controls access to DBMS information that is stored on disk, whether it is part of the database or the catalog.

Let us consider the top part of Fig. 1. first.

- ▶ It shows interfaces for the DBA staff, casual users who work with interactive interfaces to formulate queries, application programmers who create programs using some host programming languages, and parametric users who do data entry work by supplying parameters to predefined transactions.
- ▶ The DBA staff works on defining the database and tuning it by making changes to its definition using the DDL and other privileged commands.
- ▶ The DDL compiler processes schema definitions, specified in the DDL, and stores descriptions of the schemas (meta-data) in the DBMS catalog. The catalog includes information such as the names and sizes of files, names and data types of data items, storage details of each file, mapping information among schemas, and constraints.
- ▶ Casual users and persons with occasional need for information from the database interact using the **interactive query** interface in Figure 1.
- ▶ These queries are parsed and validated for correctness of the query syntax, the names of files and data elements, and so on by a **query compiler** that compiles them into an internal form. This internal query is subjected to **query**

optimization.

- ▶ Among other things, the **query optimizer** is concerned with the rearrangement and possible reordering of operations, elimination of redundancies, and use of efficient search algorithms during execution. It consults the system catalog for statistical and other physical information about the stored data and generates executable code that performs the necessary operations for the query and makes calls on the runtime processor.
- ▶ Application programmers write programs in host languages such as Java, C, or C++ that are submitted to a **precompiler**. The precompiler extracts DML commands from an application program written in a host programming language. These commands are sent to the DML compiler for compilation into object code for database access. The rest of the program is sent to the host language compiler.
- ▶ In the lower part of Figure 2.3, the **runtime database processor** executes (1) the privileged commands, (2) the executable query plans, and (3) the canned transactions with runtime parameters. It works with the **system catalog** and may update it with statistics. It also works with the **stored data manager**, which in turn uses basic operating system services for carrying out low-level input/output (read/write) operations between the disk and main memory.
- ▶ The runtime database processor handles other aspects of data transfer, such as management of buffers in the main memory. Some DBMSs have their own buffer management module whereas others depend on the OS for buffer management.
- ▶ **Concurrency control** and **backup and recovery systems** are shown separately as a module in this figure. They are integrated into the working of the runtime database processor for purposes of transaction management.
- ▶ It is common to have the **client program** that accesses the DBMS running on a separate computer or device from the computer on which the database resides. The former is called the **client computer** running DBMS client software and the

latter is called the **database server**.

- ▶ In many cases, the client accesses a middle computer, called the **application server**, which in turn accesses the database server.

DBMS ARCHITECTURE

The architecture of a DBMS determines how the data is stored, processed, and accessed by users. In this chapter, we will explore centralized and client-server DBMS architectures, along with their key features and examples for better understanding.

DBMS ARCHITECTURE:

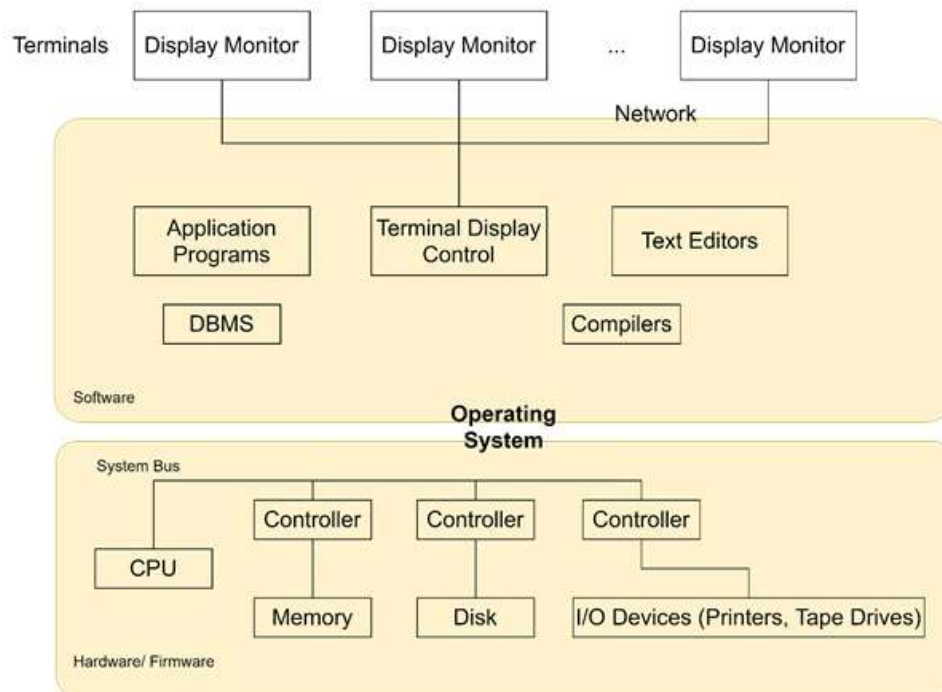
- **PHYSICAL ARCHITECTURE** (Where data lives)
 - Centralized
 - Distributed
- **APPLICATION ARCHITECTURE** (How apps connect)
 - 1-tier
 - 2-tier
 - 3-tier
- **CONCEPTUAL ARCHITECTURE** (How data is organized)
 - 3-Schema Architecture (Internal/Conceptual/External)

PHYSICAL ARCHITECTURE

Centralized DBMS Architecture

A centralized system operates through a central node. In a centralized DBMS architecture, all database operations, user interfaces, and applications are managed by a single central computer typically a **mainframe** or a **powerful server**.

This type of architecture was common in the early days of computing, when most processing power resided in a central location. Users would access the mainframe through terminals connected via a network. These terminals had minimal processing capabilities and served primarily as input/output devices.



Working of Centralized DBMS Architecture

All user interactions in a centralized DBMS take place through the terminals.

Terminals are some basic interfaces connected to the mainframe. These terminals do not process the data themselves. They send the input commands to the server. The command handles the execution. The server processes the commands and returns the results to the terminals for display.

Let's understand this with an **example**. Consider a university that uses a centralized DBMS to manage its student database. All student-related queries such as checking grades or registering for classes are processed on a central mainframe. Terminals located across the campus send requests to this central server, which processes the queries, retrieves the required data, and sends the results back to the terminals for display.

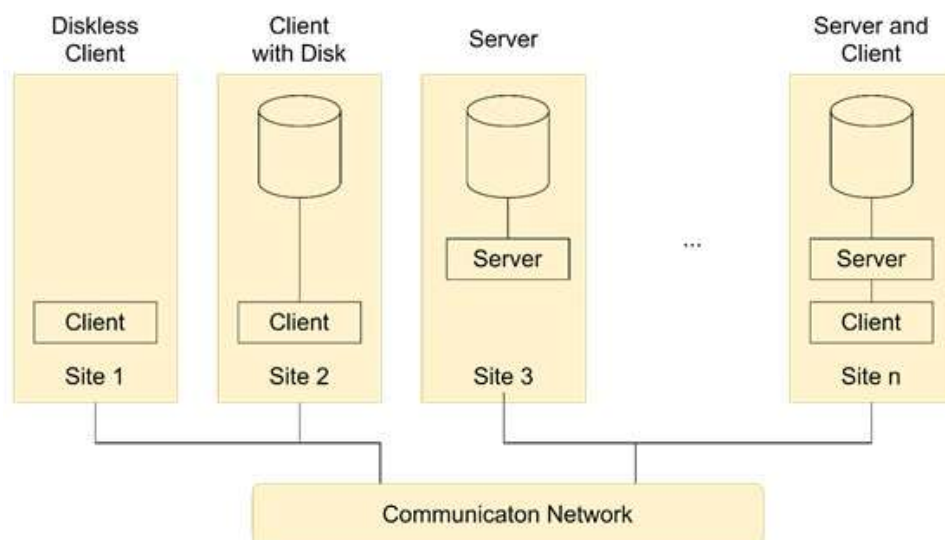
Centralized DBMS Architecture - Benefits and Drawbacks

The following table highlights the benefits and drawbacks of using a centralized DBMS architecture –

Benefits	Drawbacks
Centralized Control: The mainframe has full control over data access, making security management more straightforward.	Scalability Issues: As the number of users grows, the mainframe may struggle to handle the increased load.
Simplified Maintenance: All software updates and changes are applied to a single system. It reduces administrative overhead.	Single Point of Failure: If the mainframe fails, the entire database system becomes unavailable.

Client-Server DBMS Architecture

The client-server system is evolution of centralized system. This architecture divides the workload between clients (user-facing systems) and servers (back-end systems). The clients handle the user interface and local processing. It is the servers manage data storage, complex processing, and business logic.



Basic Structure of Client-Server Architecture

In its simplest form, the client-server model has the following objects:

- **Clients:** Machines or software applications where the users interact. These handle user

inputs and present results.

- **Servers:** Systems that store the database and execute data processing tasks.

For example, a customer service platform where employees use desktop computers (clients) to access a centralized customer database (server). The client computers send queries to the server, which processes them and returns the necessary information.

Distributing the workload between client and server helps balance resource use and reduces the load on a single system. Clients and servers can be located on different machines, facilitating remote access and distributed data processing.

Client-Server DBMS Architecture – Benefits and Drawbacks

The following table highlights the benefits and drawbacks of client-server DBMS architecture –

Benefits	Drawbacks
Simplicity – The structure is simple and straightforward and easy to implement.	Scalability Limits – As more clients connect, the server may face performance bottlenecks.
Direct Interaction – Clients interact directly with the server, resulting in faster query processing for simple tasks.	Maintenance – Each client may require updates if changes are made to the application.

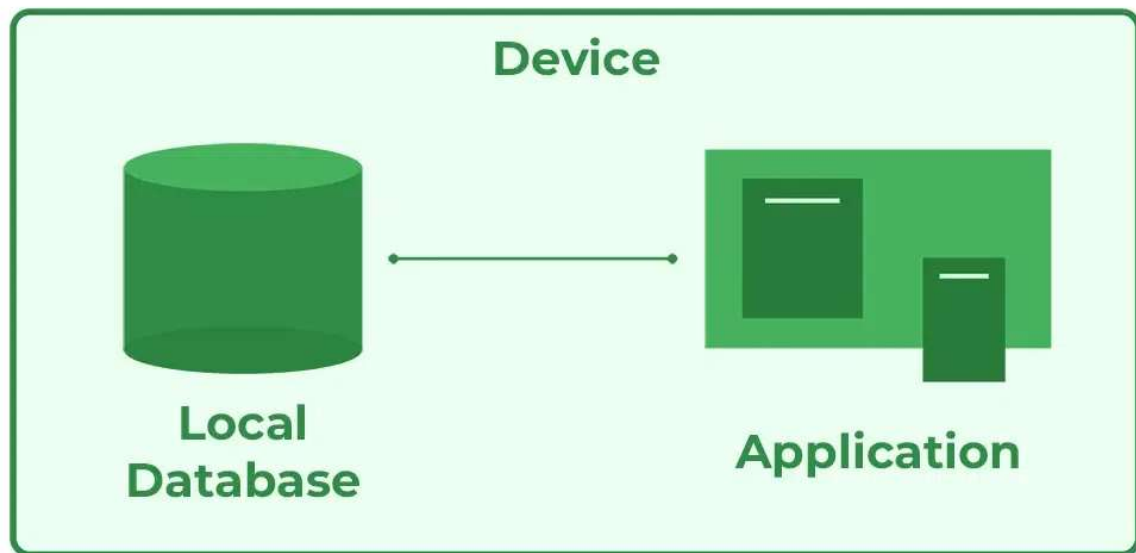
APPLICATION ARCHITECTURE

There are several types of DBMS Architecture that we use according to the usage requirements.

- 1-Tier Architecture
- 2-Tier Architecture
- 3-Tier Architecture

1-Tier Architecture

In 1-Tier Architecture, the user works directly with the [database](#) on the same system. This means the client, server and database are all in one application. The user can open the application, interact with the data and perform tasks without needing a separate server or network connection.



DBMS 1-Tier Architecture

- A common example is Microsoft Excel. Everything from the user interface to the logic and data storage happens on the same device. The user enters data, performs calculations and saves files directly on their computer.
- This setup is simple and easy to use, making it ideal for personal or standalone applications. It does not require a network or complex setup, which is why it's often used in small-scale or individual use cases.
- This architecture is simple and works well for personal, standalone applications where no external server or network connection is needed.

Advantages of 1-Tier Architecture

Below mentioned are the advantages of 1-Tier Architecture.

- ☺ **Simple Architecture:** 1-Tier Architecture is the most simple architecture to set up, as only a single machine is required to maintain it.
- ☺ **Cost-Effective:** No additional hardware is required for implementing

1-Tier Architecture, which makes it cost-effective.

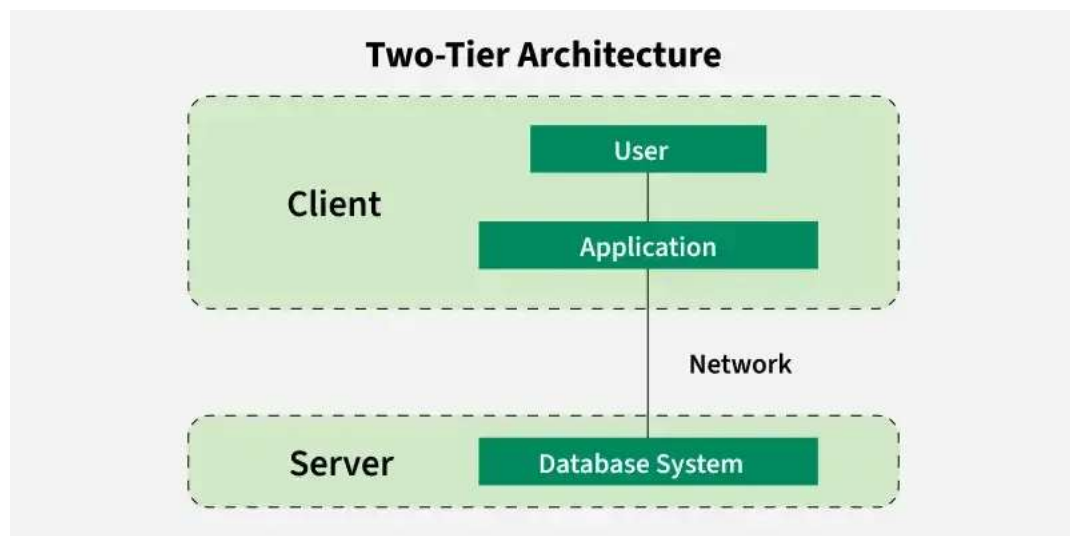
- 😊 **Easy to Implement:** 1-Tier Architecture can be easily deployed and hence it is mostly used in small projects.

Disadvantages of 1-Tier Architecture

- 😞 **Limited to Single User:** Only one person can use the application at a time. It's not designed for multiple users or teamwork.
- 😞 **Poor Security:** Since everything is on the same machine, if someone gets access to the system, they can access both the data and the application easily.
- 😞 **No Centralized Control:** Data is stored locally, so there's no central database. This makes it hard to manage or back up data across multiple devices.
- 😞 **Hard to Share Data:** Sharing data between users is difficult because everything is stored on one computer.

2-Tier Architecture

In the 2-tier architecture, the application at the client end directly communicates with the database on the server side. APIs like ODBC and JDBC are used for this interaction. The server side is responsible for providing query processing and transaction management functionalities.



DBMS 2-Tier Architecture

- On the client side, the user interfaces and application programs are run. The application on the client side establishes a connection with the

server side to communicate with the DBMS. For Example: A Library Management System used in schools or small organizations is a classic example of two-tier architecture.

- **Client Layer (Tier 1):** This is the user interface that library staff or users interact with. For example they might use a desktop application to search for books, issue them, or check due dates.
- **Database Layer (Tier 2):** The database server stores all the library records such as book details, user information and transaction logs.
- The client layer sends a request (like searching for a book) to the database layer which processes it and sends back the result. This separation allows the client to focus on the user interface, while the server handles data storage and retrieval.

Advantages of 2-Tier Architecture

- 😊 **Easy to Access:** 2-Tier Architecture makes easy access to the database, which makes fast retrieval.
- 😊 **Scalable:** We can scale the database easily, by adding clients or upgrading hardware.
- 😊 **Low Cost:** 2-Tier Architecture is cheaper than 3-Tier Architecture and Multi-Tier Architecture.
- 😊 **Easy Deployment:** 2-Tier Architecture is easier to deploy than 3-Tier Architecture.
- 😊 **Simple:** 2-Tier Architecture is easily understandable as well as simple because of only two components.

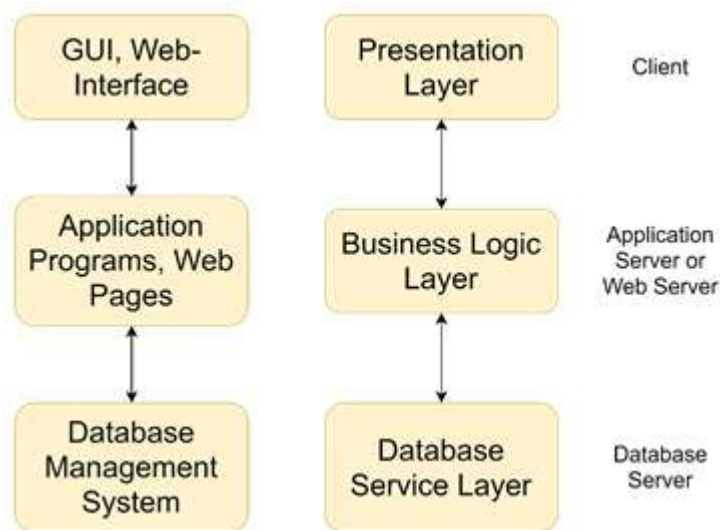
Disadvantages of 2-Tier Architecture

- 😞 **Limited Scalability:** As the number of users increases, the system performance can slow down because the server gets overloaded with too many requests.

- ☹️ **Security Issues:** Clients connect directly to the database, which can make the system more vulnerable to attacks or data leaks.
- ☹️ **Tight Coupling:** The client and the server are closely linked. If the database changes, the client application often needs to be updated too.
- ☹️ **Difficult Maintenance:** Managing updates, fixing bugs, or adding features becomes harder when the number of users or systems increases.

Three-Tier Client-Server Architecture

The speciality of three-tier architecture is that, it has an additional layer between the client and the server. This is known as application server. This intermediate layer helps manage business logic, application rules, and data processing more effectively.



Following are the **components** of three-tier client-server DBMS architecture –

- **Presentation Layer (Client)** – Displays data and collects user input.
- **Application Layer (Middle Tier)** – Processes user requests and interacts with the database.
- **Data Layer (Server)** – Handles database storage and management.

Example – A web-based ordering system for a restaurant. The client (user's web browser) interacts with a web server (application layer), which processes orders and retrieves data from the database server (data layer). This structure allows for better load distribution and supports complex business logic.

For **example**, in a healthcare information system, patient data can be accessed securely by separating user-access from direct database interaction through an application server. This improves the security by ensuring sensitive data is only accessible to authenticated users.

Another Example: E-commerce Store

- **User:** You visit an online store, search for a product and add it to your cart.
- **Processing:** The system checks if the product is in stock, calculates the total price and applies any discounts.
- **Database:** The product details, your cart and order history are stored in the database for future reference.

Advantages of 3-Tier Architecture

- 😊 **Better Load Management :** The middle tier processes requests before sending them to the database server, reducing the server's direct load.
- 😊 **Enhanced scalability:** Scalability is enhanced due to the distributed deployment of application servers. Now, individual connections need not be made between the client and server.
- 😊 **Data Integrity:** 3-Tier Architecture maintains Data Integrity. Since there is a middle layer between the client and the server, data corruption can be avoided/removed.

- 😊 **Security:** 3-Tier Architecture Improves Security. This type of model prevents direct interaction of the client with the server thereby reducing access to unauthorized data.

Disadvantages of 3-Tier Architecture

- 😞 **More Complex:** 3-Tier Architecture is more complex in comparison to 2-Tier Architecture. Communication Points are also doubled in 3-Tier Architecture.
- 😞 **Difficult to Interact:** It becomes difficult for this sort of interaction to take place due to the presence of middle layers.
- 😞 **Slower Response Time:** Since the request passes through an extra layer (application server), it may take more time to get a response compared to 2-Tier systems.
- 😞 **Higher Cost:** Setting up and maintaining three separate layers (client, server and database) requires more hardware, software and skilled people. This makes it more expensive.

Beyond Three Tiers: n-Tier DBMS Architectures

Three-tier systems are common, some applications extend it to n-tier architectures. Such architectures can include additional processing layers for more specialized tasks such as separate layers for authentication, data aggregation, or specific application services.

Example – Large enterprise applications, like those used in CRM or ERP, often utilize n-tier structures to balance tasks across various layers.

Following are the benefits of using n-tier systems –

- **Modularity** – Breaking down the application into multiple layers allows teams to

develop and manage parts independently.

- **Resilience** – Failures in one layer can be managed without affecting the entire system, which greatly enhances reliability.

CONCEPTUAL ARCHITECTURE

Three-Schema Architecture

- The Three-Schema Architecture is a well-structured framework for database management systems (DBMS) that separates the database into three distinct layers. This separation enhances flexibility, abstraction, and independence, making it easier to manage and modify the database.
- The architecture, proposed by the ANSI/SPARC committee, serves as a guide for designing and interacting with databases. Let's explore its components, advantages, and practical implications.

The three layers of this architecture are:

1. External Schema (View Level)
2. Conceptual Schema (Logical Level)
3. Internal Schema (Physical Level)

Each layer is independent of the others, ensuring data abstraction and separation of concerns.

1. **External Schema (View Level)**

What it does: Provides tailored views of the database to different users or user groups.

Purpose: Offers customized access while hiding irrelevant details.

Example: A customer sees their account balance, while a bank manager accesses detailed financial reports.

Characteristics:

- Multiple external schemas can exist.
- Security and privacy are ensured by restricting access to specific data.

2. Conceptual Schema (Logical Level)

What it does: Represents the logical structure of the entire database.

Purpose: Provides a unified view of the data for all external schemas, independent of how data is physically stored.

Example: Defines entities like Customers and Accounts with attributes and relationships.

Characteristics:

- Hides physical storage details.
- Maintains data integrity and enforces constraints.
- Focuses on logical design and relationships.

3. Internal Schema (Physical Level)

What it does: Deals with the physical storage of data on hardware.

Purpose: Optimizes storage, retrieval, and performance.

Example: Specifies file structures, indexes, and storage strategies.

Characteristics:

- Determines how data is physically stored in the database.
- Manages performance aspects like data compression and indexing.
- Illustration of the Three Layers

Real-World Analogy

Think of the architecture as a library system:

External Schema: A user interacts with a search catalog, seeing only books in their area of interest (e.g., fiction or non-fiction).

Conceptual Schema: The librarian knows the entire catalog, including all books, authors, and classifications.

Internal Schema: The library storage system determines how books are physically

arranged (e.g., by ISBN or shelf number).

Advantages of the Three-Schema Architecture

1. Data Independence:

Logical Independence: Changes to the conceptual schema do not affect external schemas.

Physical Independence: Changes to the internal schema do not affect the conceptual schema.

2. Flexibility:

Multiple external schemas allow customized views for different users.

3. Security:

Sensitive data is protected by limiting access at the external level.

4. Maintainability:

Modular design simplifies updates and system upgrades.

Data Independence

Data independence is a key benefit of this architecture:

1. Logical Data Independence:

Changes to the logical structure (e.g., adding a new entity) don't disrupt external views.

Example: Adding a new column Email in the Customer table won't affect the customer's view of their account balance.

2. Physical Data Independence:

Changes to physical storage (e.g., reorganizing files) don't impact the logical schema.

Example: Shifting from one storage format to another doesn't affect queries.

Use Cases of the Three-Schema Architecture

1. Database Development:

Ensures a clear separation between user requirements and physical implementation.

2. Data Security:

Restricts user access to only relevant information.

3. Scalability:

Facilitates system upgrades without affecting users.

Limitations of the Three-Schema Architecture

1. Complex Implementation:

Requires careful design to ensure independence between layers.

2. Overhead:

Abstraction layers may introduce additional complexity in querying and system management.

Actors on the Scene

- ▶ For a small personal database, such as the list of addresses, one person typically defines, constructs, and manipulates the database, and there is no sharing.
- ▶ However, in large organizations, many people are involved in the design, use, and maintenance of a large database with hundreds or thousands of users.
- ▶ We identify the people whose jobs involve the day-to-day use of a large database and call them the actors on the scene.
- ▶ In the next section, we'll consider people who may be called workers behind the scene—those who work to maintain the database system environment but who are not actively interested in the database contents as part of their daily job.

1. Database Administrators

- ⇒ In any organization where many people use the same resources, there is a need for a chief administrator to oversee and manage these resources.
- ⇒ In a database environment, the primary resource is the database itself, and

the secondary resource is the DBMS and related software. **Administering these resources** is the **responsibility** of the database administrator (**DBA**).

- ⇒ The DBA is responsible for authorizing access to the database, coordinating and monitoring its use, and acquiring software and hardware resources as needed.
- ⇒ The DBA is accountable for problems such as security breaches and poor system response time.
- ⇒ In large organizations, the DBA is assisted by a staff that carries out these functions.

2. Database Designers

- ⇒ Database designers are responsible for identifying the data to be stored in the database and for choosing appropriate structures to represent and store this data.
- ⇒ These tasks are mostly undertaken before the database is actually implemented and populated with data.
- ⇒ It is the responsibility of database designers to communicate with all prospective database users in order to understand their requirements and to create a design that meets these requirements.
- ⇒ In many cases, the designers are on the staff of the DBA and may be assigned other staff responsibilities after the database design is completed.
- ⇒ Database designers typically interact with each potential group of users and develop **views** of the database that meet the data and processing requirements of these groups.
- ⇒ Each view is then analyzed and integrated with the views of other user groups.
- ⇒ The final database design must be capable of supporting the requirements of all user groups.

3. End Users

- ⇒ End users are the people whose jobs require access to the database for querying, updating, and generating reports; the database primarily exists for their use.

⇒ There are several categories of end users:

■ **Casual end users** occasionally access the database, but they may need different information each time. They use a sophisticated database query interface to specify their requests and are typically middle- or high-level managers or other occasional browsers.

■ **Naive or parametric end users** make up a sizable portion of database end users. Their main job function revolves around constantly querying and updating the database, using standard types of queries and updates— called **canned transactions**—that have been carefully programmed and tested. Many of these tasks are now available as **mobile apps** for use with mobile devices. The tasks that such users perform are varied. A few examples are:

- ▶ Bank customers and tellers check account balances and post withdrawals and deposits.
- ▶ Reservation agents or customers for airlines, hotels, and car rental companies check availability for a given request and make reservations.
- ▶ Employees at receiving stations for shipping companies enter package identifications via bar codes and descriptive information through buttons to update a central database of received and in-transit packages.
- ▶ Social media users post and read items on social media Web sites.

■ **Sophisticated end users** include engineers, scientists, business analysts, and others who thoroughly familiarize themselves with the facilities of the DBMS in order to implement their own applications to meet their complex requirements.

■ **Standalone users** maintain personal databases by using ready-made program packages that provide easy-to-use menu-based or graphics-based interfaces. An example is the user of a financial software package that stores a variety of personal financial data.

⇒ A typical DBMS provides multiple facilities to access a database.

⇒ Naive end users need to learn very little about the facilities provided by the DBMS; they simply have to understand the user interfaces of the mobile

apps or standard transactions designed and implemented for their use.

- ⇒ Casual users learn only a few facilities that they may use repeatedly.
- ⇒ Sophisticated users try to learn most of the DBMS facilities in order to achieve their complex requirements.
- ⇒ Standalone users typically become very proficient in using a specific software package.

4. System Analysts and Application Programmers (Software Engineers)

- ⇒ System analysts determine the requirements of end users, especially naive and parametric end users, and develop specifications for standard canned transactions that meet these requirements.
- ⇒ **Application programmers** implement these specifications as programs; then they test, debug, document, and maintain these canned transactions.
- ⇒ Such analysts and programmers—commonly referred to as **software developers or software engineers**—should be familiar with the full range of capabilities provided by the DBMS to accomplish their tasks.

Workers behind the Scene

- ⇒ In addition to those who design, use, and administer a database, others are associated with the design, development, and operation of the DBMS software and system environment.
- ⇒ These persons are typically not interested in the database content itself.
- ⇒ We call them the workers behind the scene, and they include the following categories:

■ **DBMS system designers and implementers** design and implement the DBMS modules and interfaces as a software package. A DBMS is a very complex software system that consists of many components, or **modules**, including modules for implementing the catalog, query language processing, interface processing, accessing and buffering data, controlling concurrency, and handling data recovery and security. The DBMS must interface with other system software, such as the operating system and compilers for various programming languages.

■ **Tool developers** design and implement tools—the software packages that facilitate database modeling and design, database system design, and improved performance. Tools are optional packages that are often purchased separately. They include packages for database design, performance monitoring, natural language or graphical interfaces, prototyping, simulation, and test data generation. In many cases, independent software vendors develop and market these tools.

■ **Operators and maintenance personnel** (system administration personnel) are responsible for the actual running and maintenance of the hardware and software environment for the database system.

Although these categories of workers behind the scene are instrumental in making the database system available to end users, they typically do not use the database contents for their own purposes.

DBMS - Data Models

- ▶ It define how the logical structure of a database is modelled.
- ▶ They are fundamental entities to introduce abstraction in a DBMS.
- ▶ They define how data is connected to each other and how they are processed and stored inside the system.
- ▶ The very first data model could be flat data-models, where all the data used are to be kept in the same plane.
- ▶ Earlier data models were not so scientific, hence they were prone to introduce lots of duplication and update anomalies

Categories of Data Models

- **Conceptual (high-level, semantic) data models:** Provide concepts that are close to the way many users perceive data. (Also called entity-based or object-based data models.)
- **Physical (low-level, internal) data models:** Provide concepts that describe details of how data is stored in the computer.
- **Implementation (representational) data models:** Provide concepts that fall between the above two, balancing user views with some computer storage details.

Data Independence

Data Independence is defined as a property of DBMS that helps you to change the Database schema at one level of a database system without requiring to change the schema at the next higher level.

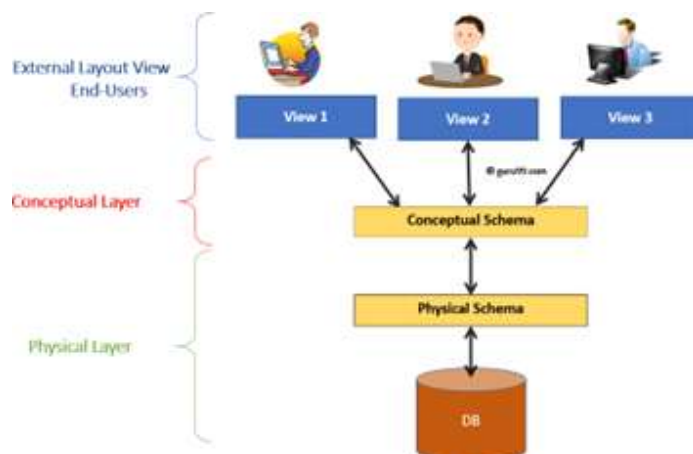
Data independence helps you to keep data separated from all programs that make use of it.

Types of Data Independence

→ Physical Data Independence

→ Logical Data Independence

Levels of Database



Consider an Example of a University Database. At the different levels this is how the implementation will look like:

Type of Schema	Implementation
External Schema	View 1: Course info(cid:int,cname:string) View 2: studeninfo(id:int, name:string)
Conceptual Shema	<pre>Students(id: int, name: string, login: string, age: integer) Courses(id: int, cname:string, credit: integer) Enrolled(id: int, grade:string)</pre>
Physical Schema	• Relations stored as unordered files. • Index on the first column of Students.

Physical Data Independence : Change internal schema without having to change conceptual schema, i.e can easily change the physical storage structures or devices with an effect on the conceptual schema.

It helps you to separate conceptual levels from the Internal/Physical levels.

It allows you to provide a logical description of the database without the need to specify physical structures.

It is achieved by the presence of the internal level of the database and then the transformation from the conceptual level of the database to the internal level.

Example of Physical Data Independence

Due to Physical independence, any of the below change will not affect the conceptual layer.

- ✓ Using a new storage device like Hard Drive or Magnetic Tapes Modifying the file organization technique in the Database Switching to different data structures.
- ✓ Changing the access method. Modifying indexes.
- ✓ Changes to compression techniques or hashing algorithms.
- ✓ Change of Location of Database from say C drive to D Drive

Logical Data Independence is the ability to change the conceptual scheme without changing :

- External views
- External API or programs

Any change made will be absorbed by the mapping between external and conceptual levels.

When compared to Physical Data independence, it is challenging to achieve logical data independence.

Due to Logical independence, any of the below change will not affect the external layer.

- ✓ Add/Modify/Delete a new attribute, entity or relationship is possible without a rewrite of existing application programs
- ✓ Merging two records into one
- ✓ Breaking an existing record into two or more records

Importance of Data Independence

- ✓ Helps you to improve the quality of the data Database system maintenance becomes affordable
- ✓ Enforcement of standards and improvement in database security You don't need to

alter data structure in application programs

- ✓ Permit developers to focus on the general structure of the Database rather than worrying about the internal implementation
- ✓ It allows you to improve state which is undamaged or undivided Database incongruity is vastly reduced.
- ✓ Easily make modifications in the physical level is needed to improve the performance of the system.

Logical Data Independence	Physical Data Independence
It is mainly concerned with the structure or changing the data definition.	Mainly concerned with the storage of the data
It is difficult as the retrieving of data is mainly dependent on the logical structure of data.	It is easy to retrieve.
You need to make changes in the Application program if new fields are added or deleted from the database.	A change in the physical level usually does not need change at the Application program level.
Compared to Logic Physical independence it is difficult to achieve logical data independence.	Compared to Logical Independence it is easy to achieve physical data independence.
Modification at the logical levels is significant whenever the logical structures of the database are changed.	Modifications made at the internal levels may or may not be needed to improve the performance of the structure.
Concerned with conceptual schema	Concerned with internal schema
Example: Add/Modify/Delete a new attribute	Example: change in compression techniques, hashing algorithms, storage devices, etc

Schema, Instance and State

- **Database Schema:** The description of a database.
- **Schema Diagram:** A diagrammatic display of (some aspects of) a database schema.
- **Schema Construct:** A component of the schema or an object within the schema, e.g., **STUDENT, COURSE**.
- **Database Instance:** The actual data stored in a database at a particular moment in time. Also called database state (or occurrence).
- **Database State:** Refers to the content of a database at a moment in time.
- **Initial Database State:** Refers to the database when it is loaded
- **Valid State:** A state that satisfies the structure and constraints of the database.
- **Distinction**
 - The database schema changes very infrequently.
 - The database state changes every time the database is updated.
 - Schema is also called **Intension**, whereas state is called **Extension**

DBMS Languages

- **Data Definition Language (DDL)** allows the DBA or user to describe and name entities, attributes, and relationships required for the application plus any associated integrity and security constraints
- **Data Manipulation Language (DML)** provides basic data manipulation operations on data held in the database
- **Data Control Language (DCL)** defines activities that are not in the categories of those for the DDL and DML, such as granting privileges to users, and defining when proposed changes to a databases should be irrevocably made.
- **Low Level or Procedural DML:** allow user to tell system exactly how to manipulate data (e.g., Network and hierarchical DMLs)
- **High Level or Non-procedural DML**(declarative language): allow user to state what data is needed rather than how it is to be retrieved
- (e.g., SQL, QBE)

Based on the data model used:

Traditional: Relational, Network, Hierarchical.

Emerging: Object-oriented, Object-relational.

Other classifications:

Single-user (typically used with micro- computers) vs. multi- user (most DBMSs).

Centralized (uses a single computer with one database) vs. distributed (uses multiple computers, multiple databases)

History of Data Models

Relational Model: proposed in 1970 by E.F. Codd (IBM), first commercial system in 1981-82. Now in several commercial products (DB2, ORACLE, SQL Server, SYBASE, INFORMIX).

Network Model: the first one to be implemented by Honeywell in 1964-65 (IDS System). Adopted heavily due to the support by CODASYL (CODASYL - DBTG report of 1971). Later implemented in a large variety of systems - IDMS (Cullinet - now CA), DMS 1100 (Unisys), IMAGE (H.P.), VAX -DBMS (Digital Equipment Corp.).

Hierarchical Data Model: implemented in a joint effort by IBM and North American Rockwell around 1965. Resulted in the IMS family of systems. The most popular model. Other system based on this model: System 2k (SAS inc.)

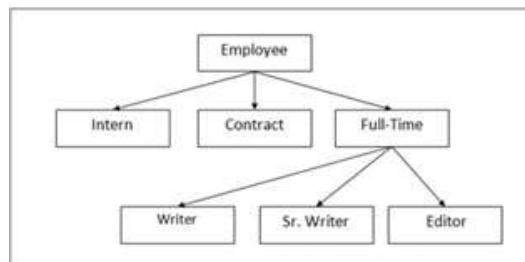
Object-oriented Data Model(s): several models have been proposed for implementing in a database system. One set comprises models of persistent O-O Programming Languages such as C++ (e.g., in OBJECTSTORE or VERSANT), and Smalltalk (e.g., in GEMSTONE).

Additionally, systems like O 2, ORION (at MCC - then ITASCA), IRIS (at H.P.- used in Open OODB).

Object-Relational Models: Most Recent Trend. Started with Informix Universal Server. Exemplified in the latest versions of Oracle-10i, DB2, and SQL Server etc. systems.

Hierarchical Data Model

- In Hierarchical Model, a hierarchical relation is formed by collection of relations and forms a tree-like structure.
- The relationship can be defined in the form of parent child type.
- One of the first and most popular Hierarchical Model is Information Management System (IMS), developed by IBM



Merits

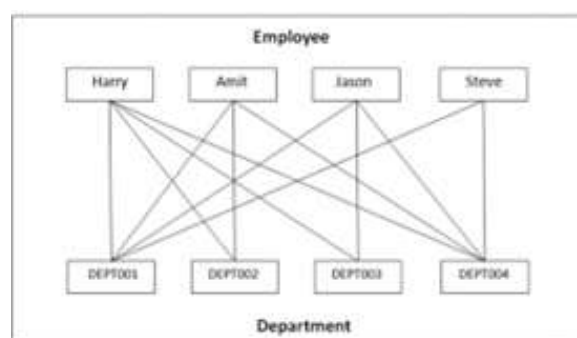
- The design of the hierarchical model is simple.
- Provides Data Integrity since it is based on parent/ child relationship Data sharing is feasible since the data is stored in a single database.
- Even for large volumes of data, this model works perfectly.

De-Merits

- Implementation is complex.
- This model has to deal with anomalies like Insert, Update and Delete.
- Maintenance is difficult since changes done in the database may want you to do changes in the entire database structure.

Network Data Model

- The Hierarchical Model creates hierarchical tree with parent/ child relationship, whereas the Network Model has graph and links.
- The relationship can be defined in the form of links and it handles
- many-to-many relations. This itself states that a record can have more than one parent.



Merits :

- Easy to design the Network Model
- The model can handle one-one, one-to-many, many-to-many relationships.

- It isolates the program from other details.
- Based on standards and conventions.

De-Merits :

- Pointers bring complexity since the records are based on pointers and graphs.
- Changes in the database isn't easy that makes it hard to achieve structural independence.

Relational Data Model

A relational model groups data into one or more tables. These tables are related to each other using common records.

The data is represented in the form of rows and columns i.e. tables:

	Column1	Column2	Column3
Row1			
Row2			
Row3			
Row4			
Row5			

Example : Let us see an example of two relations **<Employee>** and **<Department>** linked to each other, with DepartmentID, which is **Foreign Key** of **<Employee>** table and Primary key of **<Department>** table.

<Employee>		
EmployeeID	EmpName	DepartmentID
E09	Emily	D001
E04	Tom	D002
E11	Emma	D003

<Department>		
DepartmentID	DeptName	DeptZone
D001	Finance	North
D002	Operations	East
D003	Marketing	West

An arrow points from the DepartmentID column in the Employee table to the DepartmentID column in the Department table, indicating a foreign key relationship.

Merits:

- It does not have any issues that we saw in the previous two models i.e. update, insert and delete anomalies have nothing to do in this model. Changes in the database do not require you to affect the complete database.
- Implementation of a Relational Model is easy.
- To maintain a Relational Model is not a tiresome task.

De-Merits :

- Database inefficiencies hide and arise when the model has large volumes of data.
- The overheads of using relational data model come with the cost of using powerful hardware and devices.